

STE PTT 1 Question Bank

Q. 1 (2 marks)

1) Enlist any four attributes of a good software.

Ans.

1. **Correctness** – The software should perform the required functions accurately and produce correct results.
2. **Reliability** – The software should work consistently without failures or errors.
3. **Usability** – The software should be easy to learn, understand, and use.
4. **Maintainability** – The software should be easy to modify, update, and fix when requirements change.

2) Define Software Engineering. List any two categories of software.

Ans. Software Engineering is the systematic approach to the design, development, testing, deployment, and maintenance of software

It applies engineering principles to software creation to ensure : -

- i. High quality
- ii. Low cost
- iii. On-time delivery
- iv. Maintainability
- v. Scalability

Two categories of software as follows : -

1. **System Software** – Software that manages and controls computer hardware and provides a platform for other software.
Example: Operating systems like Windows, Linux.
2. **Application Software** – Software designed to perform specific tasks for users.
Example: MS Word, Web browsers, Calculator.

3) Enlist the types of software.

Ans. (Write any 2 or 4)

1. **System Software** : Manages computer hardware and provides a platform for running applications.
2. **Application Software** : Helps users perform specific tasks.
3. **Embedded Software** : Built into devices to control their specific functions.
4. **Firmware** : Low level software programmed into hardware devices.
5. **Networking Software** : Helps in managing and communicating over computer networks.
6. **Security Software** : Protects system and data from threats, viruses, malware and unauthorized access.
7. **Gaming Software** : Software designed for entertainment and playing games.
8. **Cloud Software** : Runs on remote servers and accessed via the internet.
9. **Development Software** : Used by developers to create, test and maintain software.

4) State any two principles of software deployment practice.

Ans. (Write any 2 principles)

1. Fixing bugs and Enhancing Performance : The release of updates, bug fixes, and performance enhancements is made possible by deployment. It guarantees that consumers are utilizing reliable and effective software versions.

2. Competitiveness and Time-to-Market : Time-to-market is shortened by quick and effective deployment, which helps businesses remain competitive in ever-changing markets. Having the capacity to release updates rapidly can give you a competitive edge.

3. User Experience and Satisfaction : Updates that are regularly released guarantee that consumers receive the newest enhancements, which raises user happiness. Promptly implementing deployment in response to customer input improves the overall user experience.

4. Feedback System for Ongoing Improvement : Frequent deployment establishes a feedback loop that offers insightful information about how consumers engage with the programme. This information can be utilized to guide future development and deployment strategies and promote continual improvement.

5. Optimizing Resources and Cutting Costs : Optimizing resources and cutting costs are two benefits of efficient deployment techniques. Processes for deployment can be automated to save manual labor, cut down on mistakes, and guarantee consistent outcomes in all situations

5) Define Data Modelling in Software Engineering.

Ans. Data Modelling is done in RAD (Rapid Application Development) Model which is based on Incremental Process Model in Software Engineering.

1. The information gathered in the Business Modelling phase is reviewed and analyzed to form sets of data objects vital for the business.

2. Find out what data the business uses.

3. Organize data into tables, fields, and relationships.

4. Example: Student info, course list, marks — how they are connected.

6. Define the terms related to design concepts

i. Abstraction

ii. Modularity

iii. Encapsulation

Ans.

1. Abstraction : Hiding the implementation details of a module or component and exposing only the necessary information. This makes the software more flexible and easier to change.

2. Modularity : Breaking down the software into smaller, independent, and reusable components or modules. This makes the software easier to understand, test, and maintain.

3. Encapsulation : Wrapping the data and functions of a module or component into a single unit, and providing controlled access to that unit. This helps to protect the data and functions from unauthorized access and modification.

7. Explain functional requirements.

Ans. In software engineering, "functional" refers to the specific features and behaviours a system must exhibit to perform its intended tasks and meet user needs, detailing what the software should do. These are also known as functional requirements and describe actions like data processing, user interactions, calculations, and other specific operations, forming the core functionality that developers build the system around.

8. Explain Eliciting requirements and enlist any two techniques.

Ans. The process of investigating and learning about a system's requirements from users, clients, and other stakeholders is known as requirements elicitation.

Two techniques of Eliciting requirements are as follows : -

1. **Interviews** – Directly asking users or stakeholders questions to understand their needs and expectations.
2. **Questionnaires** – Using a set of written questions to collect requirements from a large number of users.

9. What is DSDM? Mention any two features of DSDM?

Ans. DSDM is an Agile method that focuses on delivering software on time and within budget.

It is based on the idea that users should be involved all the time and that the most important work is done first.

Features of DSDM are as follows : *(Write any 2)*

1. **Users are always involved** - They give feedback and help the team
2. **Work is done in steps** - The software grows bit by bit
3. **Time and cost are fixed** - Only the most useful features are built first
4. **Testing is done at every step** - Not just at the end

Q. 2 (4 marks)

1. Explain Adaptive Software Development with respect to:

i) Focus of ASD

ii) Steps in ASD

Ans. Adaptive Software Development (ASD) is a software development methodology that emphasizes adaptability to respond to changing requirements and environments. It focuses on continuous learning and is considered a part of agile software development.

Focus of ASD : -

1.Iterative Development : ASD breaks down the project into small, manageable iterations, delivering incremental values.

2.Risk Management : ASD involves identifying and addressing risks early in the development process.

3.Continuous Testing : It integrates testing throughout the development process to ensure high-quality outcomes.

4.Decentralized Control : It encourages decision-making at the team level instead of relying on top-down directives.

5.Customer Satisfaction : It prioritizes customer satisfaction by delivering products that meet their needs and expectations.

Steps in ASD : -

1. Speculation

Start the project and prepare the initial plan.

Identify requirements, user needs, and project goals.

Divide the project into release cycles.

2. Collaboration

Encourage teamwork and communication.

Team members share ideas and solve problems together.

Members should trust, help, and respect each other.

Individual creativity is also encouraged.

3. Learning

Team members learn from development and feedback.

Improves their understanding of the project and technology.

Learning is done through:

a. Focus Groups – Get user feedback.

b. Technical Reviews – Review technical work.

c. Project Postmortem – Learn from completed work.

2. Describe phases in Spiral Model. Draw a neat-labelled diagram showing the following areas:

- (i) Customer communication**
- (ii) Planning**
- (iii) Risk Analysis**
- (iv) Engineering**
- (v) Construction and release**
- (vi) Customer evaluation**

Ans. The Spiral Model is a software development model that combines iterative development with risk analysis. The software is developed through repeated cycles called spirals. Each cycle produces a more complete version of the software.

Phases of Spiral Model :-

- 1. Customer Communication**
Requirements are collected and discussed with the customer. The objectives and expectations of the software are identified.
- 2. Planning**
The project activities, resources, schedule, and requirements for the next cycle are planned.
- 3. Risk Analysis**
Possible technical, financial, and project risks are identified and analyzed. Solutions are developed to reduce these risks.
- 4. Engineering**
The software is designed, developed, and tested according to the requirements.
- 5. Construction and Release**
The developed software is constructed, tested, and released to the customer or users.
- 6. Customer Evaluation**
The customer evaluates the software and provides feedback. This feedback is used as input for the next spiral cycle.

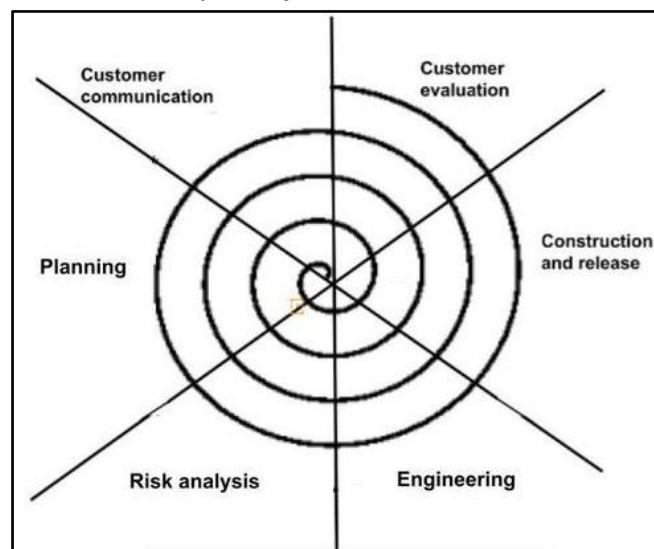


Fig : Spiral Model

3. Describe any four planning principles.

Ans. (Write any 4)

- 1. Define Project Scope :** Clearly outline the objectives, deliverables, and boundaries of the software project.
- 2. Risk Management :** Identify potential risks, assess their impact, and develop mitigation strategies.
- 3. Resource Allocation :** Determine the necessary human resources, technology, and infrastructure for the project.
- 4. Development Process :** Establish a structured approach for development, including methodologies, tools, and standards.
- 5. Testing Strategy :** Define the testing approach, including types of tests and schedule.
- 6. Communication Plan :** Ensure clear and effective communication channels among team members and stakeholders.
- 7. Iterative Planning :** Recognize that planning is an ongoing process and adjust the plan as needed.
- 8. Customer Involvement :** Engage the customer throughout the planning phase to ensure alignment with their needs.
- 9. Quality Assurance :** Implement procedures to ensure the software meets quality standards and requirements.

4. State any four Software characteristics.

Ans. (Write any 4)

1. Software is developed or engineered; it is not manufactured.
2. Software is developed using programming languages and requires continuous updates and maintenance.
3. Software can be easily changed or updated according to user needs.
4. Software systems can be highly complex with many interconnected modules.
5. Unlike hardware, software does not degrade over time but may become outdated or may contains unusual bugs.
6. Software needs hardware to run but can work across various hardware types with modifications.

5. State the core principles of Software Engineering.

- Ans. 1. Modularity :** Breaking down the software into smaller, independent, and reusable components or modules. This makes the software easier to understand, test, and maintain.
- 2. Abstraction :** Hiding the implementation details of a module or component and exposing only the necessary information. This makes the software more flexible and easier to change.
- 3. Encapsulation :** Wrapping the data and functions of a module or component into a single unit, and providing controlled access to that unit. This helps to protect the data and functions from unauthorized access and modification.
- 4. DRY principle (Don't Repeat Yourself) :** Avoiding duplication of code and data in the software. This makes the software more maintainable and less error-prone.

5. KISS principle (Keep It Simple, Stupid) : Keeping the software design and implementation as simple as possible. This makes the software more understandable, testable, and maintainable.

6. YAGNI (You Ain't Gonna Need It) : Avoiding adding unnecessary features or functionality to the software. This helps to keep the software focused on the essential requirements and makes it more maintainable.

7. SOLID principles : A set of principles that guide the design of software to make it more maintainable, reusable, and extensible. This includes the Single Responsibility Principle, Open/Closed Principle, Liskov Substitution Principle, Interface Segregation Principle, and Dependency Inversion Principle.

8. Test-driven development : Writing automated tests before writing the code, and ensuring that the code passes all tests before it is considered complete. This helps to ensure that the software meets the requirements and specifications.

6. Explain the phases of Prototyping Model with suitable diagram.

Ans. Prototyping Model has six phases as follows :

1. Requirements gathering and analysis
2. Quick design
3. Build a Prototype
4. Initial user evaluation
5. Refining Prototype
6. Implement Product and Maintain

1.Requirements Gathering and Analysis

Collect and understand the user's requirements.

2.Quick Design

Create a basic design of the system.

3.Build a Prototype

Develop a simple working model of the system.

4.Initial User Evaluation

Show the prototype to the user and collect feedback.

5.Refining the Prototype

Modify and improve the prototype based on feedback.

6.Implement Product and Maintain

Develop the final product, test it, deploy it, and maintain it.

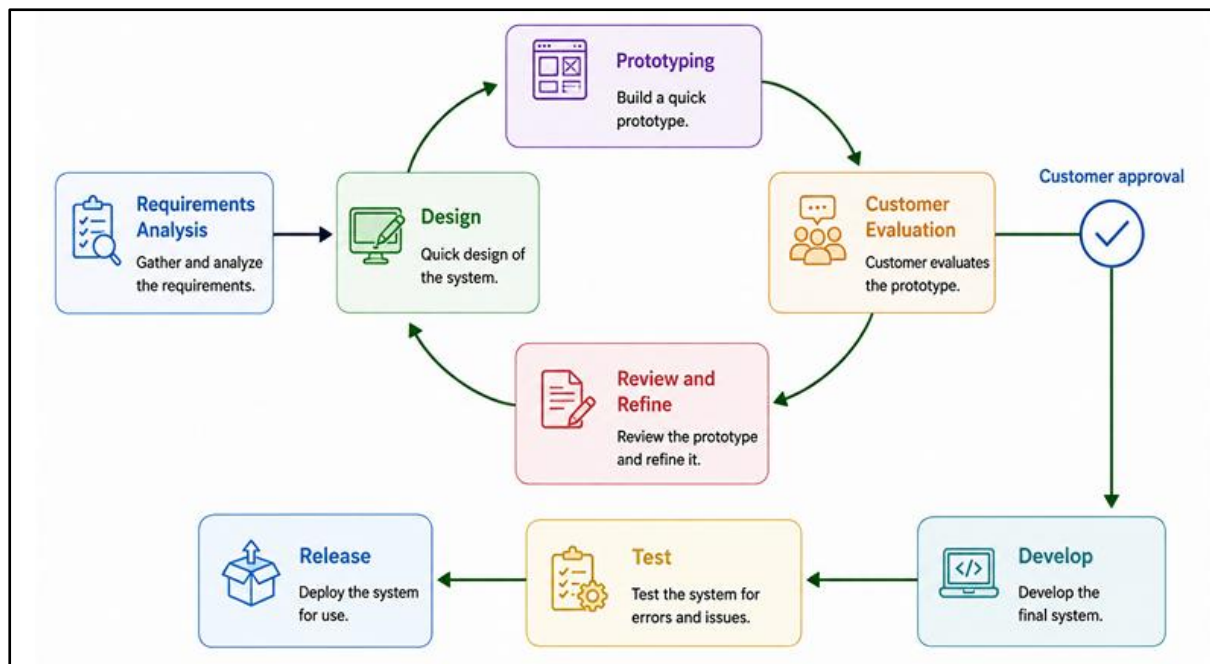


Fig : Prototyping Model

7. Describe four Modelling principles.

Ans. 1. Understanding and Communication : Models simplify complex systems, making them easier to understand and discuss among various stakeholders (developers, users, clients, etc.).

2. Analysis and Design : Models allow for analysis of the system's behaviour, performance, and potential issues before implementation, helping identify potential problems early on.

3. Guiding Development : Models act as a guide during the development process, ensuring that the final product aligns with the intended design and requirements.

4. Facilitating Collaboration : Models promote better communication and collaboration among the development team and other stakeholders by providing a shared understanding of the system.

8. Explain Extreme Programming and describe any four important practices of XP?

Ans.

Extreme Programming (XP) is one of the most popular Agile software development methodologies focused on delivering high-quality software quickly and efficiently.

1. Promotes close collaboration between developers, testers, and customers.
2. Supports iterative development with faster delivery and continuous improvement.
3. XP is a way to build software quickly and with good quality.
4. Two developers work together on one computer (this is called pair programming).
5. Before writing the actual code, they first write tests to check if the code works (test driven development).
6. The team gets feedback from the customer every day, so they can make changes quickly.
7. Best for: Projects where requirements change often.

Q. 3 (6 marks)

1. Define SRS. Describe SRS format and state any two characteristics of SRS document.

Ans. A Software Requirements Specification (SRS) format includes sections like Introduction (Purpose, Scope), Overall Description (Product Perspective, User Characteristics), Specific Requirements (Functional, Non-Functional), and Appendices. Key characteristics of a good SRS are that it must be complete, correct, consistent, unambiguous, verifiable, modifiable, and traceable. It also should be clear, testable, and design-independent, serving as a binding contract between the customer and development team.

A Software Requirements Specification (SRS) follows the IEEE 830 standard format, outlining the software's purpose, features, user interactions, and technical constraints to guide the development process. The format typically includes sections for the Introduction, which details the purpose and scope; an Overall Description of the product and its constraints; and Specific Requirements, which encompass functional, non-functional, and external interface requirements, often supported by user stories or use cases.

Two characteristics of SRS are as follows : -

- 1. Complete :** The document should include all functional and non-functional requirements, system constraints, and external interfaces, leaving no essential information or features absent.
- 2. Correct :** It must accurately reflect the actual needs and specifications of the customer and stakeholders.
- 3. Unambiguous :** Every requirement should have only one clear interpretation, avoiding vague language or confusing statements.
- 4. Consistent :** The SRS should be free of contradictions between requirements, and it should maintain uniform terminology and formatting throughout the document.

2. With the help of diagram describe Waterfall Model with its all phases. Also state its merits and demerits.

- Ans.**
1. The Waterfall Model is a linear and sequential software development process in which each phase must be completed before the next phase begins.
 2. It flows step-by-step like a waterfall, from top to bottom.
 3. The waterfall model is a traditional method, sometimes called the classic life cycle.
 4. This is one of the initial models.
 5. As the figure implies stages are cascaded and shall be developed one after the other.
 6. In other words one stage should be completed before the other begins.

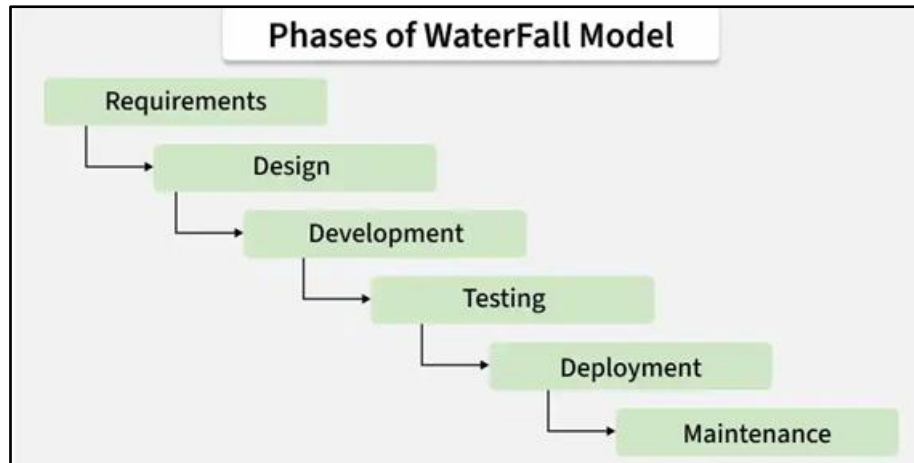


Fig : Waterfall Model

Waterfall model consists of 6 phases. They are as follows : -

- 1. Requirement Analysis :** Gather and analyze the requirements from the client or users.
- 2. System Design :** Design the system architecture and overall solution.
- 3. Development (Coding) :** Write and develop the code as per the design.
- 4. Testing :** Test the software to find and fix defects.
- 5. Deployment :** Deploy the software to the production environment.
- 6. Maintenance :** Maintain and update the software after deployment.

Merits : -

1. Simple and easy to understand.
2. Easy to manage due to its structured approach.
3. Each phase has clear deliverables and goals.
4. Documentation is maintained at every step.
5. Best suited for small projects with fixed requirements.
6. Easy to track project progress.

Demerits : -

1. Difficult to make changes once a phase is completed.
2. Not suitable for complex or long-term projects.
3. Testing is done late, so errors are found late.
4. Customer gets to see the product only at the end.
5. High risk if initial requirements are misunderstood.
6. No working software until the last phase.

3. Describe any six principles of communication practices.

Ans.

- 1. Active Listening :** Pay close attention to what others are saying, both verbally and nonverbally, to understand their perspectives and needs.
- 2. Clear and Concise Messaging :** Use simple language, avoid jargon, and structure your communication to be easily understood.
- 3. Choose the Right Medium :** Select the most appropriate communication channel (e.g., email, instant messaging, video calls, face-to-face meetings) based on the context and audience.
- 4. Feedback Culture :** Provide and receive constructive feedback regularly to improve individual and team performance.
- 5. Active Participation :** Engage actively in discussions, meetings, and brainstorming sessions.
- 6. Preparation :** Before communicating, especially in meetings, prepare talking points and gather information to ensure clarity and effectiveness.

4. Explain Rapid Application Development (RAD) Model with respect to the following:

i) Diagram

ii) Phases of RAD Model

iii) When to use RAD model

Ans. The RAD Model focuses on rapid prototyping and quick feedback over long development and testing cycles.

It aims to develop software quickly using reusable components.

Phases : -

- a. Business Modeling
- b. Data Modeling
- c. Process Modeling
- d. Application Generation
- e. Testing and Turnover

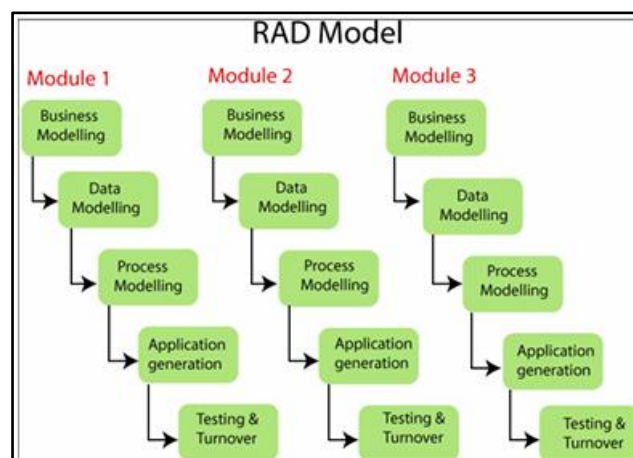


Fig : RAD Model

1) Business Modelling :

The business model for the product under development is designed in terms of flow of information and the distribution of information between various business channels.

2) Data Modelling :

The information gathered in the Business Modelling phase is reviewed and analyzed to form sets of data objects vital for the business.

3) Process Modelling :

The data object sets defined in the Data Modelling phase are converted to establish the business information flow needed to achieve specific business objectives as per the business model.

4) Application Generation :

The actual system is built and coding is done by using automation tools to convert process and data models into actual prototypes.

5) Testing & Turnover :

The overall testing time is reduced in the RAD model as the prototypes are independently tested during every iteration.

However, the data flow and the interfaces between all the components need to be thoroughly tested with complete test coverage.

RAD Model can be used when the following conditions are satisfied : -

1. Requirements are well understood and can be clearly defined.
2. The project requires quick development and delivery.
3. User involvement and feedback are easily available.
4. The system can be divided into independent modules.

5. Explain six key points of Scrum with suitable diagram.

Ans.

1. Sprint : A time-boxed period of one month or less during which a product increment is developed is known as a Sprint, and a new sprint begins immediately after the previous one ends.

2. Release : Usable product increment can be delivered based on requirements, which is referred to as a Release in Scrum.

3. Sprint Review : Completed work is presented to stakeholders for feedback and future improvements during the Sprint Review.

4. Sprint Retrospective : Continuous improvement in processes and team performance is achieved through the Sprint Retrospective, where the team reflects on the sprint.

5. Product Backlog : All features, enhancements, and requirements are organized and prioritized in the Product Backlog.

6. Sprint Backlog : Selected items from the Product Backlog, along with a plan to complete them during the sprint, make up the Sprint Backlog.

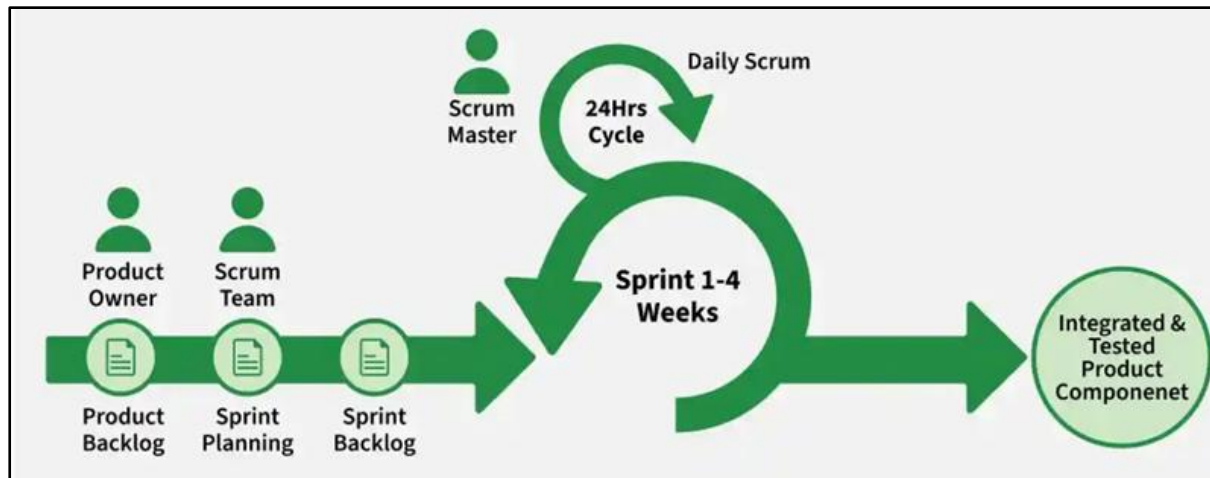


Fig : Scrum

6. Explain the steps for Developing Use Cases.

Ans.

1. Identify Actors : Determine all entities that interact with the system. Actors can be human users, other systems, or even time-based events. Categorize them as primary actors (initiating interactions) and secondary actors (responding to primary actors).

2. Identify Use Cases : For each actor, identify the specific goals they want to achieve through interacting with the system. Each goal typically corresponds to a distinct use case. Use cases should be named clearly and concisely, reflecting the user's objective (e.g., "Register for Course," "Process Order").

3. Define Use Case Descriptions : For each identified use case, create a detailed description outlining the sequence of events.

This typically includes : -

- a. Use Case Name :** A clear and concise name.
- b. Actors :** The primary and secondary actors involved. (Actors are referred as Stakeholders)
- c. Preconditions :** Conditions that must be true before the use case can begin.
- d. Trigger :** The event that initiates the use case.
- e. Basic Flow (Main Scenario) :** The ideal, successful sequence of steps from the actor's perspective.
- f. Alternate Flows (Extensions) :** Variations or alternative paths that can occur, including error handling or optional steps.
- g. Postconditions :** The state of the system after the use case is successfully completed.